Práctica 2 — Parte 1: Virtualización de servicios web con Vagrant

Administración y Gestión de Redes (AGR)

Pablo Elvira Calatayud

Índice

- 1. Objetivo
- 2. Configuración proporcionada por el script
- 3. Pruebas realizadas
- 4. Estudio de consumo del servicio virtualizado
  - 4.1. Memoria
  - 4.2. CPU
  - 4.3. Disco
  - 4.4. Estimación: red virtualizada del laboratorio 3103
  - 4.5. Comparación con servidores comerciales
  - 4.6. Conclusiones
- 5. Problemas encontrados en el anfitrión

1. Objetivo

Desplegar de forma automática, con un script Vagrant (Ruby), una máquina virtual Linux que ejecute el servidor web Node.js SSR-master-server, accesible desde la propia máquina en el puerto **3000** y desde el exterior en el puerto **8080**, y estudiar el consumo de recursos del servicio virtualizado.

Entregable: `parte1-vagrant/Vagrantfile`.

2. Configuración proporcionada por el script

Elemento	Configuración
Proveedor / box	VirtualBox, `bento/ubuntu-22.04` (clon enlazado)
Recursos	1 vCPU (tope del 80 % de una CPU del anfitrión), 1024 MB RAM; parametrizables con `VM_CPUS` y `VM_MEMORY`
Nombre	`agr-nodeserver` (VM y hostname)
Red 1 – NAT	Salida a Internet + *port forwarding* `anfitrión:8080 → VM:3000` en todas las interfaces del anfitrión
Red 2 – privada	host-only `192.168.56.10/24` (red del servicio)
Red 3 – privada	host-only `192.168.57.10/24` (red de gestión)
Red 4 – pública (opcional)	bridge sobre una interfaz física, IP por DHCP: `BRIDGE_IF=<if> vagrant up`
Sistema	Teclado español (consola y X11), locale `es_ES.UTF-8`, zona horaria `Europe/Madrid`
Software	Node.js 22 (NodeSource), código clonado de GitHub en `/opt/ssr-master-server`
Servicio	Unidad systemd `ssr-server` (usuario sin privilegios `nodeapp`, reinicio automático)
Seguridad	Cortafuegos `ufw`: sólo 22/tcp y 3000/tcp
Inicio de sesión	`/etc/profile.d/99-servidor-info.sh` muestra sistema, red, estado del servicio, URLs, puertos y cortafuegos

El aprovisionamiento se divide en cuatro *provisioners* de tipo `shell` (`sistema`, `node`, `servidor`, `login-info`) idempotentes, por lo que `vagrant provision` puede repetirse sin efectos indeseados.

Incidencia en el servidor. `app.js` ya ejecuta `app.listen(3000)` y `bin/www` (lo que lanza `npm start`) vuelve a escuchar en el mismo puerto, por lo que `npm start` termina con `EADDRINUSE`. El servicio lanza directamente `node app.js`.

3. Pruebas realizadas

Prueba	Orden	Resultado
Servicio desde la VM	`vagrant ssh -c "curl -s localhost:3000"`	HTML de Data-Logger
Port forwarding	`curl http://localhost:8080` (anfitrión)	HTTP 200
Red privada	`curl http://192.168.56.10:3000`	HTTP 200
Escritura de datos	`curl "localhost:8080/record?id_nodo=n1&temperatura=24.5&..."`	`Saving: ...` y CSV en `/logs`
Carga	20 000 + 5000 peticiones a `/record`	53 CSV generados, sin errores
Información al iniciar sesión	`vagrant ssh`	Se imprime la configuración

4. Estudio de consumo del servicio virtualizado

Medidas tomadas sobre la VM `agr-nodeserver` desplegada con el `Vagrantfile` (Ubuntu 22.04, 1 vCPU, 1 GB RAM, Node.js 22 + SSR-master-server). Los datos en bruto están en `doc/development/parte1/consumo_raw.txt`.

Anfitrión: AMD Ryzen 7 7700X (8 núcleos / 16 hilos), 30 GiB RAM, Ubuntu 24.04, VirtualBox 7.2.20, Vagrant 2.4.9.

Metodología: medidas desde el anfitrión (`ps`, `VBoxManage metrics`, `du`) y desde el invitado (`free`, `df`, `top`), en reposo y durante una carga de 5000 peticiones HTTP secuenciales a `/record` a través del port forwarding (`localhost:8080 → VM:3000`). Antes se habían enviado ~20 000 peticiones. El crecimiento de la imagen se vigiló con `watch "ls -al ~/VirtualBox\ VMs/agr-nodeserver/Snapshots/"`.

4.1. Memoria

Medida	Reposo	Con carga	Tras carga
RSS del proceso `VBoxHeadless` (anfitrión)	**1.04 GiB**	1.05 GiB	1.05 GiB
RAM usada por el invitado (sin caché)	202 MiB	206 MiB	204 MiB
Caché de disco del invitado	685 MiB	680 MiB	692 MiB
RSS del proceso `node app.js`	85 MiB	86 MiB	—

- La aplicación es ligera: el SO + servidor solo usan **~200 MiB**, y Node ~85 MiB.
- Sin embargo, al anfitrión **la VM le cuesta toda la RAM asignada (1 GB) más ~70 MiB de sobrecarga de VirtualBox**: el invitado usa la memoria libre como caché de

disco y VirtualBox no la devuelve. En virtualización completa se paga la RAM *reservada*, no la *necesaria*.
- La carga apenas mueve la memoria (+10 MiB): el servidor no acumula estado.
- Conclusión: podría asignarse **512 MiB** sin pérdida de servicio y reducir el coste a la mitad.

4.2. CPU

Medida	Reposo	Con carga
CPU del anfitrión usada por la VM	~0.06 %	**~2.3 %** (≈ 0.37 hilos de 16)
CPU del invitado (user + kernel)	0 %	~24 % de 1 vCPU
`node` en `top`	0.5 %	26.7 %

- En reposo la VM es prácticamente gratuita en CPU.
- Con carga se sirvieron **5000 peticiones en 40.9 s (~122 pet/s, ~8 ms por petición)**. El cuello de botella no es el servidor (vCPU al 24 %) sino el cliente: un `curl` nuevo por petición, en serie. Con clientes concurrentes la VM admitiría bastante más tráfico antes de saturar su vCPU.
- Parte del coste es de kernel (≈ 1/3), debido a la red virtual NAT y a la escritura en disco de cada petición.

4.3. Disco

Elemento	Tamaño
Box base `bento/ubuntu-22.04` (`~/.vagrant.d/boxes`)	811 MiB
Disco diferencial de la VM (clon enlazado, `Snapshots/*.vmdk`)	**2.1 GiB**
Uso dentro del invitado	5.5 GiB de 31 GiB virtuales
Logs CSV tras ~25 000 registros	1.2 MiB (53 ficheros)

- Gracias a `linked_clone` la VM no copia la imagen base: solo guarda las diferencias (paquetes, Node.js, `node_modules`, logs). Una segunda VM igual costaría otros ~2 GiB, no 5.5.
- El disco es *dinámico*: los 31 GiB son un máximo, el fichero crece al escribirse bloques nuevos. Durante la carga de 5000 peticiones el `.vmdk` no cambió de forma visible (2.1G): los datos generados (~0.4 MiB, ≈ 50–80 B por registro) son despreciables frente a la granularidad de bloques del disco virtual. Al ritmo medido harían falta ~1 M de registros para crecer ~50–80 MiB.
- Si los datos se borran dentro de la VM el `.vmdk` **no** encoge; habría que compactarlo (`VBoxManage modifymedium --compact`).

4.4. Estimación: red virtualizada del laboratorio 3103

Según el plano (`doc/development/enunciado/plan_numeracion_3103.pdf`) el laboratorio tiene **4 routers** (RA–RD), **2 servidores** (SRV1 y SRV2) en 10.100.0.0/24, el PC del profesor y **4 redes**

de 8 PC (32 PC). Se consideran dos escenarios:

- **Réplica completa:** 4 routers + 2 servidores + 32 PC = **38 VMs** (el PC del profesor es el propio anfitrión).
- **Réplica reducida** (la desplegada en la parte 2, que el enunciado permite): 4 routers + 1 servidor + 1 PC por red = **9 VMs**.

Coste por VM tomado de las medidas: **~1.07 GiB de RAM** en el anfitrión con 1 GB asignado (≈ RAM asignada + 70 MiB de sobrecarga), ~0.06 % de CPU en reposo y ~2 GiB de disco diferencial (+ 811 MiB de imagen base compartida). En la versión optimizada se asignan 256 MiB a routers y PC (≈ 0.32 GiB reales) y 512 MiB a los servidores (≈ 0.57 GiB).

Escenario	VMs	RAM anfitrión	Disco (clones enlazados)	CPU en reposo
Completa, 1 GB por VM	38	**~41 GiB**	~77 GiB	~2 %
Completa, optimizada	38	~12.7 GiB	~40 GiB	~2 %
Reducida, 1 GB por VM	9	~9.6 GiB	~19 GiB	< 1 %
Reducida, optimizada	9	~3.1 GiB	~10 GiB	< 1 %

- La réplica completa sin optimizar **no cabe** en el anfitrión de prácticas (30 GiB de RAM); optimizada ocupa ~42 % de la RAM. La reducida cabe de sobra.
- **La RAM es el recurso limitante**, no la CPU: un router o un PC del laboratorio está ocioso casi siempre.
- El disco crece con cada VM sólo por sus cambios: con imágenes `qcow2` con *backing file* (equivalente KVM de los clones enlazados) los routers y PC, que apenas escriben, ocupan poco más que la imagen base compartida.
- Con KVM (parte 2) KSM (*Kernel Same-page Merging*) permite además que VMs con el mismo sistema compartan páginas de memoria idénticas.

4.5. Comparación con servidores comerciales

Un servidor de rack actual de 2 sockets (p. ej. AMD EPYC 9005 o Intel Xeon 6) ofrece del orden de **hasta 384 núcleos / 768 hilos y 1–6 TB de RAM**.

Con las cifras medidas, para un servidor de 1 TB de RAM y 768 hilos:

Límite	Cálculo	VMs como esta
RAM (dejando ~10 % al hipervisor)	920 GiB / 1.07 GiB	**~860**
CPU con carga como la del ensayo	768 hilos / 0.37 hilos	~2000
CPU en reposo	—	prácticamente ilimitado

- De nuevo, la RAM manda: ese servidor alojaría **~22 réplicas completas** del laboratorio (38 VMs) sin optimizar, o **~70** con la configuración optimizada; de la réplica reducida (9 VMs), ~95 y ~300 respectivamente.

- Frente al anfitrión de prácticas (16 hilos, 30 GiB), el servidor comercial tiene ~48× hilos y ~33× RAM.
- Para un servicio así de ligero (85 MiB de Node) una VM completa desperdicia memoria: con **contenedores** (Docker/LXC), que comparten el kernel del anfitrión, cada instancia costaría ~100 MiB en lugar de ~1 GiB, es decir, ~10× más densidad. Las VMs compensan cuando se necesita aislamiento fuerte o un kernel/SO distinto (p. ej. routers con su propia pila de red, como en la segunda parte).

4.6. Conclusiones

1. El servidor Node.js necesita ~200 MiB de RAM y muy poca CPU; la VM en cambio consume toda la RAM asignada en el anfitrión.
2. El dimensionado importa: bajar a 512 MiB reduce el coste a la mitad sin afectar al servicio.
3. Los clones enlazados y los discos dinámicos reducen mucho el disco ocupado; los datos de la aplicación son despreciables.
4. Para replicar redes completas, la RAM es el cuello de botella; la CPU sólo importa con carga real.

5. Problemas encontrados en el anfitrión

Problema	Causa	Solución
`No usable default provider`	El módulo `vboxdrv` no estaba cargado	Ver filas siguientes
VirtualBox 7.1 no carga el módulo	Kernel 7.0 demasiado nuevo para 7.1	Actualizar a VirtualBox 7.2
`modprobe vboxdrv` rechazado	Secure Boot activo y módulo firmado con clave no registrada	Desactivar Secure Boot (o registrar la clave MOK)
`VERR_VMX_IN_VMX_ROOT_MODE` / `VERR_SVM_IN_USE`	KVM ocupa VT-x/AMD-V en kernels recientes	`sudo modprobe -r kvm_amd kvm` antes de `vagrant up`